CNN Evaluation and Transfer Learning

Structured Notes

Custom CNN Training, Model

Evaluation, Confusion Matrix,
Error Analysis, Data Augmentation, and ResNet-18

Contents

1 Training a Custom CNN on Real Data **2**
1.1 Loading Pre-Processed Data . . . 2
1.2 Accuracy Calculation . . . 2
1.3 Training Strategy . . . 2

2 Hyperparameter Tuning and Debugging **3**
2.1 Key Hyperparameters . . . 3
2.2 Debugging Dimension Mismatches . . . 3
2.2.1 Common Error: Size Mismatch in Fully Connected Layer . . . 3
2.3 CNN Training Results . . . 4

3 Understanding Overfitting **4**
3.1 Detecting Overfitting from Training Curves . . . 4
3.2 Data Size vs. Model Depth . . . 4

4 Model Evaluation on Test Data **5**
4.1 Test Set Evaluation . . . 5
4.2 Confusion Matrix . . . 5
4.3 Key Evaluation Metrics . . . 5

5 Analyzing Wrong Predictions **6**
5.1 Visualizing Correct and Incorrect Predictions . . . 6
5.2 Common Causes of Misclassification . . . 6
5.3 Real-World ML Workflow . . . 6

6 Limitations of Custom CNN **7**

7 Transfer Learning **7**
7.1 Why Transfer Learning Works . . . 7
7.2 Two Approaches to Transfer Learning . . . 7
7.3 Transfer Learning Strategy with ResNet-18 . . . 7

8 Data Augmentation **8**
8.1 Common Augmentation Techniques . . . 8
8.2 Why a Custom Dataset Class is Needed . . . 8

9 ResNet-18 Implementation **8**
9.1 ImageNet Normalization . . . 8
9.2 Image Size for ResNet-18 . . . 8
9.3 Transform Pipelines . . . 9
9.4 ResNet-18 Results . . . 9

10 Memory Management for Large Datasets **9**

11 Summary **10**

1 Training a Custom CNN on Real Data

CNN Training Pipeline

Training a Convolutional Neural Network involves iteratively updating model parameters through forward passes, loss computation, backpropagation, and weight updates. The process is monitored by tracking training loss, validation loss, training accuracy, and validation accuracy across epochs.

1.1 Loading Pre-Processed Data

When working with image datasets, pre-processed data can be saved as `.pt` (PyTorch tensor) files to avoid repeating the pre-processing pipeline every time. This saves significant time:

- Load the saved tensor file directly
- No re-processing of raw images required
- Only run the cells that are strictly needed

Time-Saving Tip

If the data has already been pre-processed and saved, there is no need to run the entire pre-processing pipeline again. Simply load the saved `.pt` file and proceed to model definition and training.

1.2 Accuracy Calculation

The accuracy function compares model predictions against true labels:

Accuracy Calculation

```
def calculate_accuracy(outputs, labels):
    predictions = torch.argmax(outputs, dim=1)
    correct = (predictions == labels).sum().item()
    return correct / len(labels)
```

Listing 1: Custom Accuracy Function

Accuracy Formula:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Samples}}$$

The function works as follows:

1. Get predictions using `torch.argmax` on the output logits
2. Compare predictions with actual labels element-wise
3. Sum the number of correct matches
4. Divide by the total number of labels to get accuracy

This accuracy function is also available directly in libraries like scikit-learn, but understanding how it works internally is important.

1.3 Training Strategy

Training and Validation Phases

Training Phase:

1. Forward pass through the network

2. Compute the loss
3. Backpropagation to calculate gradients
4. Update weights using the optimizer

Validation Phase:

1. No gradient updates (use `torch.no_grad()`)
2. Evaluate model performance directly on the validation set
3. Monitor overfitting and generalization

The training loop tracks four key metrics across epochs:

- Training loss
- Validation loss
- Training accuracy
- Validation accuracy

Gradient Reset

At the start of every epoch, reset the running metrics (training loss, training accuracy) to zero. The gradients must be zeroed out before each batch to prevent gradient accumulation from previous iterations.

2 Hyperparameter Tuning and Debugging

2.1 Key Hyperparameters

The following hyperparameters were adjusted during the CNN training process:

Hyperparameter	Initial Value	Adjusted Value
Batch Size	32	64
Dropout	0.5	0.3
Learning Rate	0.01	0.001
Epochs	10	30

Learning Rate Sensitivity

If the learning rate is too high, the validation loss will oscillate or increase instead of converging. Reducing the learning rate from 0.01 to 0.001 produced significantly smoother and better convergence behavior.

2.2 Debugging Dimension Mismatches

When modifying CNN architecture (e.g., changing the number of filters per layer), dimension mismatches are a common error.

2.2.1 Common Error: Size Mismatch in Fully Connected Layer

When changing convolutional layers from `(16, 32, 64)` to `(32, 64, 128)`, the fully connected layer input size must be updated accordingly.

FC Layer Input Size Calculation:

$$\text{FC input} = \text{last_conv_channels} \times \text{spatial_height} \times \text{spatial_width}$$

For example, with an input image of 128×128 and three convolutional layers with max pooling:

- After conv1 + pool: channels $\times$ 64 $\times$ 64
- After conv2 + pool: channels $\times$ 32 $\times$ 32
- After conv3 + pool: channels $\times$ 16 $\times$ 16

If the last conv layer outputs 128 channels: FC input $= 128 \times 16 \times 16$

Pooling and Spatial Dimensions

Pooling layers reduce the height and width of the feature maps but do not change the number of channels. The channel count is determined by the convolutional layer filters. The fully connected layer input must match the flattened output of the last convolutional/pooling layer.

2.3 CNN Training Results

With a 3-layer custom CNN on a binary classification task (cat vs. dog):

- Training accuracy reached **87%**
- Validation accuracy reached **79–80%**
- Test accuracy: approximately **80%**
- Optimal stopping point: around epoch 20–21

3 Understanding Overfitting

Overfitting Scenarios

There are three key scenarios when comparing training and validation performance:

Scenario	Indicators	Diagnosis
Overfitting	Training accuracy high, validation accuracy low	Model memorizes training data
Underfitting	Both training and validation accuracy low	Model too simple
Good Model	Both training and validation accuracy high	Model generalizes well

3.1 Detecting Overfitting from Training Curves

Overfitting can be detected by observing the training curves:

- When validation loss stops decreasing and starts increasing while training loss continues to decrease, the model is overfitting
- The ideal stopping point is where the validation loss reaches its minimum (plateau) before it starts increasing
- An 8% gap between training and validation accuracy is still acceptable in practice

Key Insight: Trust Validation Over Training

Always trust the validation accuracy over training accuracy. **Generalization is more important than training performance.** The validation set represents how the model performs on unseen data, which is the actual goal.

3.2 Data Size vs. Model Depth

Relationship between data size and model complexity:

- Too much data for a shallow model → overfitting
- A 3-layer CNN trained on 8,000 images can overfit because the data volume is too high for such a shallow architecture
- Reducing images from 8,000 to 4,000 can produce a better fit for a shallow model
- For deep learning: large datasets require deep models

4 Model Evaluation on Test Data

Why Evaluate on Test Data

Training tells whether the model is learning, but evaluation tells whether the model performs correctly on unseen data. The test dataset must **never** be used during training. It serves as the final, unbiased assessment of model performance.

4.1 Test Set Evaluation

After training and validation, the model is evaluated on the held-out test set:

- Test accuracy was approximately **80%** with the custom CNN
- This was consistent with the validation accuracy, indicating the model generalizes well
- For a small custom CNN, this is considered good performance

4.2 Confusion Matrix

Confusion Matrix

A confusion matrix shows the distribution of correct and incorrect predictions across all classes. For binary classification (cat vs. dog):

	Predicted Cat	Predicted Dog
Actual Cat	True Positive (TP)	False Negative (FN)
Actual Dog	False Positive (FP)	True Negative (TN)

Computing Confusion Matrix

```python
from sklearn.metrics import confusion_matrix
import seaborn as sns

# Compute confusion matrix
cm = confusion_matrix(all_labels, all_predictions)

# Visualize
sns.heatmap(cm, annot=True, fmt='d',
            xticklabels=['Cat', 'Dog'],
            yticklabels=['Cat', 'Dog'])
```

Listing 2: Confusion Matrix with sklearn and Seaborn

A perfect confusion matrix (all predictions on the diagonal) would require 100% accuracy and precision, which is a dream scenario that should never realistically be expected.

4.3 Key Evaluation Metrics

Precision: Fraction of correct positive predictions.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Why accuracy alone is not enough:

- Accuracy does not show error distribution or class-wise performance
- A model could have high overall accuracy but perform poorly on one class
- The confusion matrix reveals which specific types of errors the model makes

5 Analyzing Wrong Predictions

Error Analysis

Model errors are more valuable than correct predictions. By analyzing misclassified images, one can identify the specific weaknesses of the model and determine areas for improvement.

5.1 Visualizing Correct and Incorrect Predictions

Identifying Wrong Predictions

```
wrong_images = []
wrong_labels = []

for i in range(len(predictions)):
    if predictions[i] != labels[i]:
        wrong_images.append(images[i])
        wrong_labels.append(labels[i])
```

Listing 3: Filtering Wrong Predictions

5.2 Common Causes of Misclassification

Misclassified images typically share certain characteristics:

- **Blurry images:** Lack of clear features for the filters to detect
- **Small features:** Subject occupies a small portion of the image
- **Noisy backgrounds:** Multiple objects or complex backgrounds confuse the model
- **Similar postures:** A dog sitting in a posture typical of cats can confuse edge detection
- **Multiple subjects:** Images containing both a cat and a dog
- **Non-target objects:** Images containing humans or irrelevant objects

Key Insight: Errors Are More Valuable Than Correct Predictions

The errors a model makes are always more valuable than its correct predictions. Based on error analysis, one can:

- Optimize the model architecture
- Identify key areas requiring improvement
- Improve the training data (e.g., background removal, data cleaning)

5.3 Real-World ML Workflow

The complete ML workflow in industry follows this cycle:

1. **Train** the model
2. **Analyze errors** — examine what the model gets wrong and why
3. **Improve** the data or model based on findings
4. Repeat until satisfactory performance is achieved

Background removal was highlighted as an important data preprocessing step: removing irrelevant background information helps the model focus on the actual subject features.

6 Limitations of Custom CNN

While the custom 3-layer CNN achieved 80% accuracy, this has fundamental limitations:

- **80% accuracy** is good but not sufficient for production use
- The model was trained **from scratch** with limited data
- The CNN had to learn **edges, textures, and shapes** all from zero
- Limited model depth restricts the complexity of features it can learn

The Key Question

Can knowledge be reused instead of learning everything from scratch? Just as a person who already knows what animals look like does not start from zero when asked to classify cats versus dogs, a model can leverage previously learned features.

7 Transfer Learning

Transfer Learning

Transfer learning is the technique of using a pre-trained model (trained on a large dataset like ImageNet) and adapting it to a new, typically smaller, task. Instead of learning features from scratch, the model reuses knowledge it has already acquired.

7.1 Why Transfer Learning Works

- Pre-trained models have already learned general features (edges, textures, shapes) from millions of images
- Saves computational resources and training time
- Works especially well when the target dataset is small
- Leverages features learned from large-scale datasets like ImageNet

7.2 Two Approaches to Transfer Learning

Aspect	Feature Extraction	Fine-Tuning
Strategy	Freeze most layers, train only the classifier head	Train deeper layers along with the classifier
When to use	Small dataset, similar domain	Larger dataset or different domain
Layers modified	Only the final fully connected layer(s)	Final layers + some deeper convolutional layers
Training time	Faster	Slower

7.3 Transfer Learning Strategy with ResNet-18

The implementation follows four steps:

1. **Load pre-trained model:** Load ResNet-18 with ImageNet weights
2. **Freeze feature extractor:** Freeze all convolutional layers so their weights are not updated
3. **Replace the final layer:** Replace the last fully connected layer to match the number of target classes
4. **Train on the target dataset:** Train only the new classifier layer on the target data

8 Data Augmentation

Data Augmentation

Data augmentation is the technique of artificially creating image variations during training to improve model generalization. No new data points are created — the same dataset is presented in randomized variations (flips, rotations, crops). This helps the model see richer versions of the data.

Augmentation Applies Only to Training Data

Data augmentation is applied **only** to the training set. Validation and test data must remain clean (only resizing and normalization) to provide an unbiased evaluation of model performance.

8.1 Common Augmentation Techniques

- **Random Horizontal Flip** ($p = 0.5$): Randomly flips images horizontally
- **Random Rotation** (± 10 degrees): Rotates images clockwise or anti-clockwise by up to 10 degrees
- **Random Vertical Flip**: Flips images vertically
- **Random Crop**: Randomly crops a portion of the image
- **Random Brightness/Contrast**: Adjusts brightness and contrast randomly

8.2 Why a Custom Dataset Class is Needed

- Previously, pre-processed tensors were saved as `.pt` files
- With augmentation, transformations must happen **dynamically** — differently each time an image is loaded
- Therefore, images are loaded directly from file paths and transforms are applied on-the-fly
- Once transformed, the files can be saved as `.pt`, but these files are much larger due to the pixel transformations

9 ResNet-18 Implementation

9.1 ImageNet Normalization

ResNet-18 was pre-trained on ImageNet, so inputs must be normalized using ImageNet statistics:

$$\text{mean} = [0.485, 0.456, 0.406], \quad \text{std} = [0.229, 0.224, 0.225]$$

These values are available in the PyTorch `torchvision` library and on the ImageNet documentation. Using these specific normalization values ensures the input distribution matches what ResNet-18 was trained on.

9.2 Image Size for ResNet-18

ResNet-18 was originally trained on 224×224 images. All input images must be resized to this dimension.

9.3 Transform Pipelines

Transform Pipelines for ResNet-18

```python
import torchvision.transforms as transforms

IMG_SIZE = 224
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD = [0.229, 0.224, 0.225]

# Training transform (with augmentation)
train_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.RandomHorizontalFlip(0.5),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD)
])

# Validation/Test transform (clean, no augmentation)
val_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD)
])
```

Listing 4: Training and Validation Transforms

9.4 ResNet-18 Results

- ResNet-18 with transfer learning achieved **98% accuracy**
- This was achieved in just **5 epochs** with batch size 32
- Compared to the custom CNN's 81%, this is a dramatic improvement
- The pre-trained model leverages features already learned from ImageNet's millions of images

Metric	Custom CNN	ResNet-18 (Transfer)
Accuracy	~81%	~98%
Epochs Required	30	5
Architecture	3 conv layers (from scratch)	Pre-trained + fine-tuned classifier
Training Data	8,000 images	8,000 images (with augmentation)

10 Memory Management for Large Datasets

When working with large datasets, memory management becomes critical:

- A system with 16 GB RAM has only about 8 GB available (45–50% is occupied by the OS)
- Loading a dataset larger than available RAM causes an **Out of Memory** error
- Solution: break the dataset into smaller `.pt` files (e.g., 2 GB chunks)
- Example: 80 GB of images converted to 200 GB of transformed tensors, broken into 2 GB files (approximately 100 files)
- This ensures batches fit within available RAM during training

Hardware Considerations

It is not about the total hardware capacity but about the **available RAM**. If the batch size

> produces data that exceeds available memory, reduce the batch size or break the dataset into smaller chunks for sequential loading.

11 Summary

- A custom 3-layer CNN can achieve ∼81% accuracy on binary image classification, but is limited by learning features from scratch
- **Hyperparameter tuning** (learning rate, batch size, dropout, epochs) significantly impacts model performance
- **Dimension mismatches** between conv layers and FC layers are a common debugging challenge — the FC input size depends on the last conv output channels and the spatial dimensions after pooling
- **Overfitting** is detected when training accuracy increases but validation accuracy decreases; always trust validation over training metrics
- **Model evaluation** requires a separate test set that was never used during training
- **Error analysis** (examining wrong predictions) is more valuable than examining correct predictions — it reveals blurry images, noisy backgrounds, and ambiguous subjects
- The real-world ML workflow is: Train → Analyze Errors → Improve Data/Model
- **Transfer learning** with ResNet-18 dramatically improves accuracy from 81% to 98% by reusing features learned on ImageNet
- **Data augmentation** (flips, rotations) applied only to training data helps models generalize better
- A good ML engineer investigates errors, not just accuracy